```
<ul>
  {% for item in ['Jess', 'Anna', 'Paul'] %}
    {{ _self.listItem(item) }}
  {% endfor %}
</ul>
```

The text between the {% macro %} and {% endmacro %} tags will now be inserted wherever it is referenced in the rest of the template. Macros are called in a function-like way, via the magic _self variable, and they behave like functions in that they have scoping rules: they don't have access to the rest of the template's data, and any variables defined inside the macro with {% set %} are unavailable outside. Data has to be explicitly passed using parameters/arguments and can even have default values defined – e.g., {% macro listItem(text = 'not set') %}.

Importing Macros

Macros are always local to the template in which they are defined; they are not available in included templates (or child templates, which we'll get to next). It would be an enormous pain to have to define the same macro in every template it is used in, but fortunately the {% import %} tag means you only have to define the macro once:

```
{# views/inc/macro.html.twig #}
{% macro listItem(text) %}
    <li>{{ text }}</li>
{% endmacro %}
```

```
{# views/macro-demo2.html.twig #}
{% import 'inc/macro.html.twig' as macros %}
<ul>
    {% for item in ['Jess', 'Anna', 'Paul'] %}
        {{ macros.listItem(item) }}
    {% endfor %}
</ul>
```